Cursada 2025
Fundamentos de la Computación Cuántica
Entrega Final



Alumno

Roumec, Iñaki: iroumec@alumnos.exa.unicen.edu.ar.

Profesores

Leiva, Lucas: lleiva@labset.exa.unicen.edu.ar.

Vázquez, Martín: martin.o.vazquez@gmail.com.

Introducción **3**
Resolución **4**
Transformada Cuántica de Fourier Inversa 4
Multiplicador 5
Sumador 5
Lógica 6
Implementación 7
Simulación y Resultados 8
Conclusiones **11**
Referencias **12**

Introducción

A lo largo de este informe, se presenta el proceso y los resultados correspondientes a la resolución del ejercicio 4 del trabajo práctico N.° 4 “Transformada Cuántica de Fourier”, dado en el curso. La consigna de este es la siguiente:

Implementar y analizar en Qiskit un multiplicador entre dos números binarios codificados en 3 qubits (mediante estados básicos), basado en dominio de Fourier. Debe implementarse, para ello, la transformada cuántica de Fourier inversa completa.

En este informe no se adjunta el código correspondiente a la entrega. Este, junto a sus pasos de ejecución, puede hallarse en la sección *releases* en el siguiente repositorio de GitHub: https://github.com/iroumec/Quantum-Computing/releases.

Aclaración: los resultados de las tablas presentadas en el informe pueden variar levemente entre las ejecuciones del código, debido a la naturaleza de las simulaciones. No obstante, estas variaciones no invalidan de ninguna manera las conclusiones presentadas en este informe.

Resolución

Transformada Cuántica de Fourier Inversa

Previo a la resolución del multiplicador, se debe implementar la transformada cuántica de Fourier inversa. Sin embargo, previo a su implementación, se debe analizar: ¿cuántos valores deberá decodificar la IQFT?

El máximo número representable por un operando de 3 qubits es 7 (**`111`**). Por consiguiente, el máximo valor obtenible de la multiplicación de dos operandos de tres qubits es 7 x 7 = 49. El número 49 puede descomponerse en potencias de dos como:

`1*2^5 (32) + 1*2^4 (16) + 0*2^3 + 0*2^2 + 0*2^1 + 1*2^0 (1)`

Lo que genera el siguiente número binario: **`110001`**. Por consiguiente, la transformada cuántica de Fourier inversa debe plantearse considerando que la cantidad de qubits a decodificar es seis. Con esto en cuenta, se presenta la implementación del circuito:

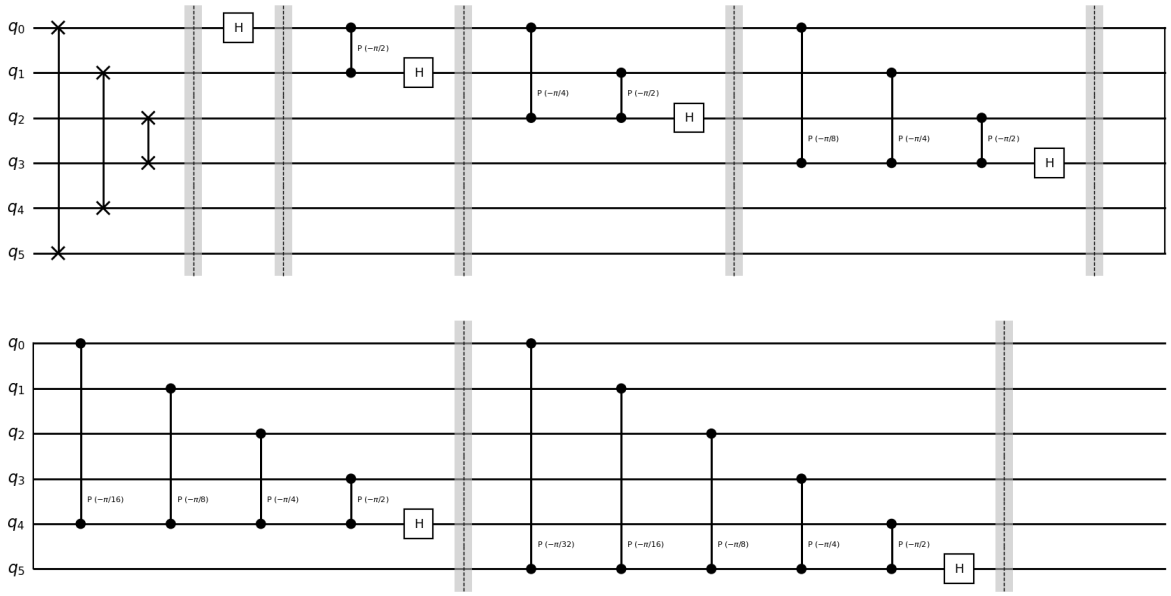


Imagen 1: circuito correspondiente a la transformada cuántica de Fourier inversa para 6 qubits. El circuito en la fila inferior es la continuación del circuito en la fila superior.

Se utiliza la compuerta **`cp`** en lugar de **`crz`**, puesto a que esta última, en Qiskit, introduce fases adicionales locales sobre el qubit objetivo, porque se implementa como: **`Rz(θ/2)•CZ•Rz(-θ/2)`**. Es decir, no es un *control-phase* puro, mientras que **`CP`** sí lo es.

Con la transformada cuántica de Fourier inversa planteada, se prosigue con el planteo del multiplicador.

Multiplicador

Sumador

Una multiplicación no son más que múltiples sumas. Por consiguiente, para la implementación del multiplicador es esencial, en primer lugar, implementar un sumador.

La implementación del sumador (y, por lo tanto, también del multiplicador) se basa en el **Teorema de Traslación** (*Shift Theorem*) o **Propiedad de Desplazamiento** (*Shift Property*). El teorema, en palabras simples, establece que:

> *"Un desplazamiento en el dominio del tiempo (o posición) equivale a una rotación de fase lineal en el dominio de la frecuencia."*

En el mundo de la computación cuántica, la implementación específica de este principio para construir un sumador se llama **Sumador QFT de Draper** (propuesto por Thomas Draper en el año 2000) [2 y 3].

Considerando esto, la suma en el dominio de Fourier puede realizarse a través de rotaciones. Por ejemplo, para sumar 1: en el bit menos significativo, debe realizarse una rotación de **π** sobre el eje Z; en el segundo bit menos significativo, una rotación de **π/2** sobre el eje Z; en el tercero, una de **π/4**; y así sucesivamente, siempre utilizando potencias de 2.

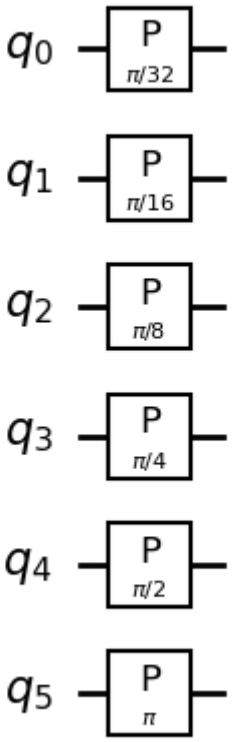


Imagen 2: circuito correspondiente a las rotaciones necesarias para sumar 1.

La pregunta ahora es: ¿cómo extender el circuito para sumar una constante *N*?

De acuerdo a la **Propiedad de Desplazamiento de la Transformada de Fourier**, la adición de un entero *N* en la base computacional corresponde a una rotación de fase lineal en la base de Fourier. Dado que la fase es aditiva, la rotación necesaria para sumar el valor *N* es simplemente *N* veces el ángulo de rotación utilizado para sumar 1 [4].

Como resultado, si se desea sumar, por ejemplo, cuatro, el circuito resultante sería el mismo que se mostró anteriormente, con la diferencia de que las rotaciones fueron multiplicadas por cuatro:

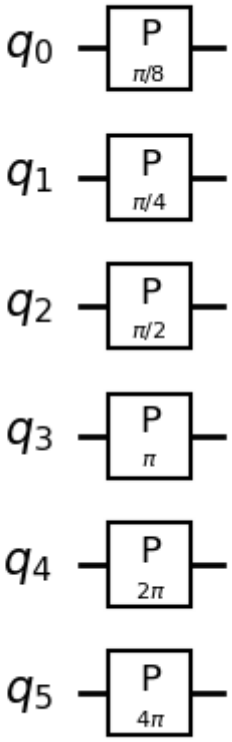


Imagen 3: circuito correspondiente a las rotaciones necesarias para sumar 4. Notar que el circuito es el mismo al presentado anteriormente, con la distinción de que las rotaciones están multiplicadas por 4.

Con la lógica de los sumadores establecida, se prosigue con la lógica del multiplicador.

Lógica

La idea de la multiplicación es similar a cómo se realiza la multiplicación en decimal. Considere la siguiente multiplicación en decimal como ejemplo:

`11 x 12`

El proceso consiste en tomar el número menos significativo del primer operando (**1**) y multiplicarlo por el número del segundo operando (**2**). Luego, el número menos significativo del primer número se multiplica por el segundo número del segundo operando (**1**). Sin embargo, el número no se multiplica simplemente por **1**, sino que, al corresponder este a las decenas, se lo está multiplicando por **`1 x 10 = 10`**.

La idea en binario es similar. De acuerdo a los pesos de los qubits y su valor, se determina el valor a sumar (o rotaciones a efectuar, en el contexto). Al trabajar con qubits cuyo valor medible varía entre **|0⟩** y **|1⟩**, el circuito se puede pensar como una secuencia de selecciones que determinan si se debe sumar o no el resultado de la multiplicación de los pesos de los qubits que se multiplican, siempre y cuando estos últimos sean ambos **|1⟩**. Por ejemplo, al multiplicar los qubits menos significativos de ambos operandos, el valor que se debe sumar es **2^0 x 2^0 = 1**. Por otro lado, si se multiplica el segundo bit menos significativo del primer operando (cuyo peso es **2^1**) por el tercero menos significativo del segundo operando (cuyo peso es **2^2**), el valor que debe sumarse es **2 x 4 = 8**.

Implementación

Si se plasma la lógica descrita anteriormente en un circuito, se obtiene el siguiente resultado:

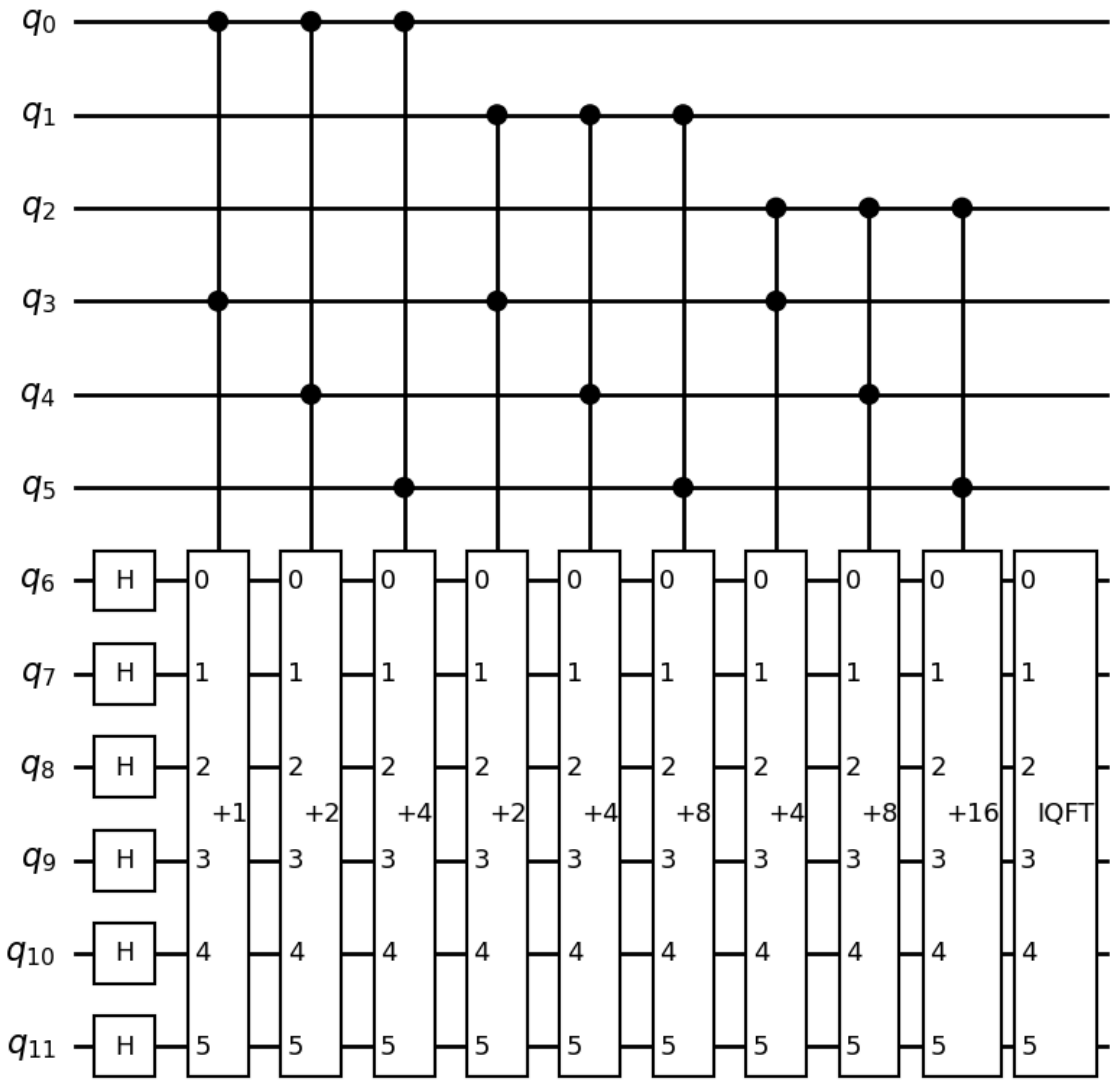


Imagen 4: circuito correspondiente a un multiplicador de operandos de 3 qubits.

Simulación y Resultados

Para corroborar el correcto funcionamiento del circuito, se colocaron en superposición los qubits de los operandos y se midieron. A su vez, también se realizó la medición de los valores decodificados.

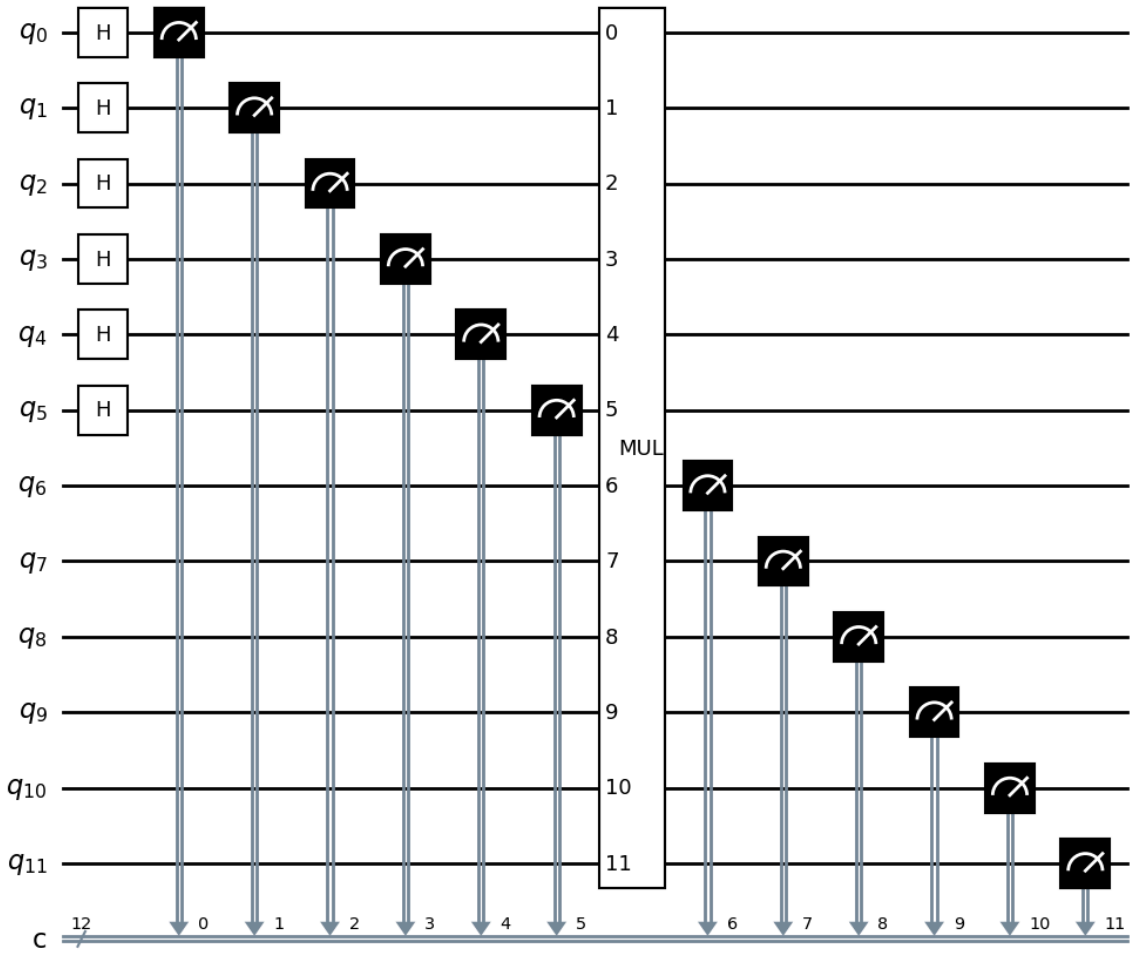


Imagen 5: circuito correspondiente a un multiplicador de operandos de 3 qubits con mediciones añadidas y entradas en superposición.

A continuación, se realizó una simulación con 10240 *shots*. Los resultados obtenidos pueden observarse en las siguientes tablas.

En estas, es posible observar:

- En la primera y segunda columnas, los valores binarios y decimales (entre paréntesis) de los operandos.
- En la tercera columna, el valor binario y decimal (entre paréntesis) del resultado.
- En la cuarta columna, el valor esperado.
- En la quinta columna, la frecuencia con la que se dio dicha combinación.
- Y en la sexta columna, si el valor obtenido coincide con el esperado.

B	A	Salida	Esperado	Frecuencia	¿Correcto?
000 (0)	000 (0)	000000 (0)	0	173	SÍ
000 (0)	001 (1)	000000 (0)	0	146	SÍ
000 (0)	010 (2)	000000 (0)	0	140	SÍ
000 (0)	011 (3)	000000 (0)	0	148	SÍ
000 (0)	100 (4)	000000 (0)	0	162	SÍ
000 (0)	101 (5)	000000 (0)	0	151	SÍ
000 (0)	110 (6)	000000 (0)	0	161	SÍ
000 (0)	111 (7)	000000 (0)	0	150	SÍ
001 (1)	000 (0)	000000 (0)	0	156	SÍ
010 (2)	000 (0)	000000 (0)	0	162	SÍ
011 (3)	000 (0)	000000 (0)	0	130	SÍ
100 (4)	000 (0)	000000 (0)	0	164	SÍ
101 (5)	000 (0)	000000 (0)	0	164	SÍ
110 (6)	000 (0)	000000 (0)	0	144	SÍ
111 (7)	000 (0)	000000 (0)	0	147	SÍ
001 (1)	001 (1)	000001 (1)	1	134	SÍ
001 (1)	010 (2)	000010 (2)	2	147	SÍ
010 (2)	001 (1)	000010 (2)	2	177	SÍ
001 (1)	011 (3)	000011 (3)	3	171	SÍ
011 (3)	001 (1)	000011 (3)	3	168	SÍ
001 (1)	100 (4)	000100 (4)	4	156	SÍ
010 (2)	010 (2)	000100 (4)	4	144	SÍ
100 (4)	001 (1)	000100 (4)	4	176	SÍ
001 (1)	101 (5)	000101 (5)	5	150	SÍ
101 (5)	001 (1)	000101 (5)	5	162	SÍ
001 (1)	110 (6)	000110 (6)	6	168	SÍ
010 (2)	011 (3)	000110 (6)	6	186	SÍ
011 (3)	010 (2)	000110 (6)	6	160	SÍ
110 (6)	001 (1)	000110 (6)	6	185	SÍ
001 (1)	111 (7)	000111 (7)	7	143	SÍ
111 (7)	001 (1)	000111 (7)	7	167	SÍ
010 (2)	100 (4)	001000 (8)	8	145	SÍ

Imagen 6: resultados correspondientes a las primeras 32 combinaciones. Como puede apreciarse, el valor esperado coincidió en todas ellas con el valor obtenido.

B	A	Salida	Esperado	Frecuencia	¿Correcto?
100 (4)	010 (2)	001000 (8)	8	146	SÍ
011 (3)	011 (3)	001001 (9)	9	160	SÍ
010 (2)	101 (5)	001010 (10)	10	151	SÍ
101 (5)	010 (2)	001010 (10)	10	156	SÍ
010 (2)	110 (6)	001100 (12)	12	176	SÍ
011 (3)	100 (4)	001100 (12)	12	139	SÍ
100 (4)	011 (3)	001100 (12)	12	174	SÍ
110 (6)	010 (2)	001100 (12)	12	184	SÍ
010 (2)	111 (7)	001110 (14)	14	187	SÍ
111 (7)	010 (2)	001110 (14)	14	155	SÍ
011 (3)	101 (5)	001111 (15)	15	161	SÍ
101 (5)	011 (3)	001111 (15)	15	162	SÍ
100 (4)	100 (4)	010000 (16)	16	177	SÍ
011 (3)	110 (6)	010010 (18)	18	189	SÍ
110 (6)	011 (3)	010010 (18)	18	151	SÍ
100 (4)	101 (5)	010100 (20)	20	169	SÍ
101 (5)	100 (4)	010100 (20)	20	176	SÍ
011 (3)	111 (7)	010101 (21)	21	182	SÍ
111 (7)	011 (3)	010101 (21)	21	148	SÍ
100 (4)	110 (6)	011000 (24)	24	162	SÍ
110 (6)	100 (4)	011000 (24)	24	174	SÍ
101 (5)	101 (5)	011001 (25)	25	166	SÍ
100 (4)	111 (7)	011100 (28)	28	154	SÍ
111 (7)	100 (4)	011100 (28)	28	146	SÍ
101 (5)	110 (6)	011110 (30)	30	188	SÍ
110 (6)	101 (5)	011110 (30)	30	147	SÍ
101 (5)	111 (7)	100011 (35)	35	133	SÍ
111 (7)	101 (5)	100011 (35)	35	145	SÍ
110 (6)	110 (6)	100100 (36)	36	176	SÍ
110 (6)	111 (7)	101010 (42)	42	157	SÍ
111 (7)	110 (6)	101010 (42)	42	166	SÍ
111 (7)	111 (7)	110001 (49)	49	146	SÍ

Imagen 7: resultados correspondientes a las últimas 32 combinaciones. Como puede apreciarse, el valor esperado coincidió en todas ellas con el valor obtenido.

Quedan demostrados así el correcto funcionamiento e implementación del circuito solicitado.

Conclusiones

A lo largo de la resolución de la entrega, no solo se asentaron los conocimientos vistos en la materia, sino que se refrescaron las habilidades de programación en Python, adquiridas durante varias de las materias de la carrera.

Junto a esta entrega, se finaliza la cursada. Esta última proporcionó una introducción a un campo emergente que, a primera instancia, puede resultar pavoroso; al mismo tiempo que forjó criterio, administrando información sobre los alcances reales de la computación cuántica y herramientas para evitar caer en fraudes o embaucadores.

Referencias

[1] "CPhaseGate — Qiskit Documentation", IBM Quantum, [Online]. Disponible en: https://quantum.cloud.ibm.com/docs/es/api/qiskit/qiskit.circuit.library.CPhaseGate (consultado: 8 dic. 2025).

[2] "DraperQFTAdder — Qiskit Documentation", IBM Quantum, [Online]. Disponible en: https://quantum.cloud.ibm.com/docs/es/api/qiskit/qiskit.circuit.library.DraperQFTAdder (consultado: 8 dic. 2025).

[3] Thomas G. Draper, "Addition on a Quantum Computer," arXiv:quant-ph/0008033, 2000. [Online]. Disponible en: https://arxiv.org/pdf/quant-ph/0008033 (consultado: 8 dic. 2025).

[4] "Propiedades de la Transformada de Fourier Continua – M2U2Propiedades_TCF," Proyecto Señales y Sistemas, Universidad Javeriana Cali, [Online]. Disponible en: https://proyectos.javerianacali.edu.co/cursos_virtuales/pregrado/senales_y_sistemas/Lecturas/Modulo2/Unidad2/M2U2Propiedades_TCF.pdf (consultado: 8 dic. 2025).